Multi-Class Sentiment Classification in Social Media Using Classical Machine Learning and Sparse Text Representations

***Abstract*—This study addresses text-based emotion recognition using the GoEmotions dataset. Although emotion detection is inherently a multi-label problem, it was simplified into a three-class sentiment classification task (positive, negative, and ambiguous) to improve interpretability and computational efficiency. Experiments with retaining neutral and mixed classes resulted in significantly lower performance due to increased ambiguity, and were therefore excluded. Exploratory data analysis revealed strong class imbalance and a long-tail distribution of emotion labels. To address these challenges, text preprocessing techniques including basic cleaning, stemming, and lemmatization were evaluated, where basic cleaning combined with stemming provided the best results. Feature extraction was performed using Bag-of-Words (BoW) and TF-IDF representations with n-gram ranges of $(1, 3)$ to capture contextual information. Class imbalance was further mitigated using the Synthetic Minority Over-sampling Technique (SMOTE). Several classical machine learning models, including Logistic Regression, Random Forest, XGBoost, SGD, and KNN, were evaluated. The best performance was achieved using TF-IDF with Random Forest, obtaining an accuracy of 0.744 and macro F1-score of 0.694. While n-gram features and SMOTE improved class representation, they did not consistently improve overall performance compared to simpler configurations. These results demonstrate that classical machine learning methods, combined with appropriate preprocessing and feature engineering, provide an effective and computationally efficient solution for emotion classification in short and noisy social media text.**

***Index Terms*—Text-based Emotion Detection, Sentiment Analysis, Natural Language Processing, GoEmotions, Classical Machine Learning, Feature Extraction, Random Forest, TF-IDF, Bag-of-Words, Preprocessing Techniques, Imbalanced Data, Short Text Classification**

I. Introduction

Text-based emotion detection is a rapidly growing research area in the field of Natural Language Processing (NLP). While traditional sentiment analysis methods classify emotions into positive, negative, and neutral categories, modern methods focus on identifying subtle emotions such as joy, anger, sadness, and fear. This helps to understand human communication in depth and develop context-aware systems [1], [2].

Nowadays, a large amount of textual data is generated through channels such as social media, online forums, and messaging platforms. This data is challenging to use directly due to informality, noise, and context-dependency. Features such as slang, abbreviations, spelling errors, and grammar inconsistencies complicate the preprocessing and feature extraction process [3], [4].

A major challenge in emotion detection is its multi-label nature. Since a single sentence can express multiple emotions, it is more complex than traditional classification problems. For example, a sentence like "I'm happy it's over, but also a bit sad" can express both joy and sadness. Therefore, it is important to identify label interdependencies [5], [6].

In addition, since language is context-sensitive and ambiguous, expressions such as sarcasm and irony can affect model performance. Deep learning and large language models are increasingly being used to address these problems [7]. Furthermore, class imbalance and long-tail distributions are observed in the dataset, which can increase model bias [3].

Given these challenges, traditional machine learning methods such as Logistic Regression and Random Forest, along with feature extraction techniques like Bag-of-Words (BoW) and TF-IDF, can be employed to represent textual data effectively. Although these are computationally efficient and interpretable, semantic understanding is limited. Although deep learning models increase semantic understanding, they also increase computational cost [1], [3].

In this research, emotions were classified as positive, negative, and ambiguous to simplify the multi-label emotion detection problem used the problem as sentiment classification. Also, performance comparisons were made using EDA, preprocessing techniques (cleaning, stemming, etc.), feature extraction (BoW, TF-IDF), and machine learning models (Logistic Regression, Random Forest, XGBoost, SGD, KNN). This study aims to identify the most effective combination of preprocessing pipeline and model.

II. Literature Review

Text-based Emotion Detection (TBED) is a rapidly growing research area in the field of Natural Language Processing (NLP). While the field was initially based on simple sentiment classifications such as positive, negative, and neutral, recent research has expanded to include more complex emotion categories such as joy, anger, fear, sadness, and surprise [2]. Deep learning approaches have made important contributions to improving semantic and contextual understanding in this field [1].

A major contribution to this field is the GoEmotions dataset introduced by Demir et al. [8]. This is a multi-label dataset with 27 fine-grained emotion labels and a neutral class, and is used as a benchmark for noisy social media text analysis. Even advanced models such as BERT on this dataset have shown

the complexity of the emotion detection problem by providing relatively low F1-scores (0.46) [9], [10].

In recent research, deep learning models have shown superior results for emotion detection. Poria et al. [1] improved emotion recognition using contextual embeddings, and Ramakrishnan [9] achieved 85.2% accuracy and 0.83 macro-F1 score by fine-tuning BERT. A comparative analysis by Kumar and Sharma [11] also showed that although deep learning models provide superior performance, classical models (SVM, Logistic Regression) provide strong baseline performance with TF-IDF features.

A major challenge in emotion detection is its multi-label nature. When more than one emotion is present in a text, it is necessary to identify the interdependencies of emotions [5]. Li et al. [6] have shown that emotion relationship graphs can be used to improve the performance of these dependency models. Also, contextual ambiguity, such as sarcasm and irony, makes emotion prediction even more difficult [7].

Another major problem is class imbalance and long-tail distribution. As Nobari et al. [3] show, neutral and common emotions are abundant, while rare emotions such as grief and pride are scarce. This leads to increased model bias. Song [10] also demonstrated that rare emotion detection is an extremely challenging problem. Balanced learning strategies are needed to solve this problem.

Preprocessing and feature engineering are very important due to the noise (slang, abbreviations, spelling errors) in social media texts [4]. Feature representation methods such as TF-IDF and Bag-of-Words (BoW) have been shown to be effective for classical models [3]. Overall, the literature shows that although deep learning models provide superior performance, classical machine learning models are still important for practical solutions due to computational efficiency and interpretability. Accordingly, this research aims to simplify the emotion space and develop a classical machine learning-based efficient sentiment classification framework.

III. Methodology

This section describes the method used to develop a system for text-based sentiment classification using classical machine learning methods. This system workflow consists of primary steps, including data preprocessing, target label transformation, feature extraction, model training, and performance evaluation. All stages are planned by considering the class imbalance, noisy text, and multi-label nature that were identified through exploratory data analysis (EDA).

Initially, this problem was considered a multi-label classification problem using fine-grained emotion labels in the dataset. This problem has become complex because there are 27 emotion categories, and some of the emotions overlap. To simplify the problem, emotions were mapped into three categories: positive, negative, and ambiguous. Neutral and mixed samples were removed to reduce noise and improve class separability.

After that, the textual data was transformed into numerical features using preprocessing techniques. Vectorization methods such as Bag-of-Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF) have been utilized. Next, performance metrics were measured using trained machine learning models. Finally, a comparative analysis was conducted to identify the best combination of preprocessing, feature extraction, and model selection.

A. Data Preprocessing

Data preprocessing is an important step in text-based sentiment classification, especially when working with noisy and informal text such as social media data. The purpose at this stage is to clean and normalize text data and preserve important information for emotion classification.

The preprocessing process begins with basic text cleaning. All text data was transformed into lowercase to ensure the dataset's consistency. Non-alphabetic characters such as punctuation and numbers were eliminated using regular expressions. This helps to reduce noise and standardize data. Also, auxiliary verbs (for example, is, are, was, have, do) were removed from the texts. These words often do not contribute much to emotion meaning, so they increase redundancy in the feature space.

Next, experiments have been conducted on various preprocessing techniques such as stopword removal, stemming and lemmatization.

Stopword Removal: Stopwords are commonly used high frequency words such as the, is, and, and in, which generally carry limited semantic meaning. Removing these words helps reduce noise and dimensionality in the feature space. However, in short and informal texts, stopwords may contribute to contextual meaning, and their removal can sometimes lead to loss of important information.

Stemming: Stemming is a text normalization technique that reduces words to their root form by removing suffixes. For example, running, runs, and runner may all be reduced to run. This process helps reduce vocabulary size and improves generalization, but it may produce non-dictionary words.

Lemmatization: Lemmatization is a more advanced normalization technique that converts words into their base or dictionary form (lemma). Unlike stemming, it considers the grammatical context of the word. For example, better is converted to good. Although more accurate than stemming, it is computationally more expensive.

An experimental comparison was conducted to investigate the performance differences between these preprocessing methods. These results were included in Section IV-B.

1) Preprocessing Pipeline Selection: To identify the best combination of text preprocessing methods, experiments have been performed on different preprocessing pipelines. Specifically, several pipelines were created combining methods such as basic cleaning, stopword removal, stemming, and lemmatization in various ways.

To compare the performance of these pipelines, TF-IDF vectorization and Random Forest classifier were used and evaluated using metrics such as accuracy, precision, and F1-

score. The results of this comparative analysis are shown in Table I.

According to Table I, the pipeline using stemming has the highest performance, with accuracy, precision, recall, and F1-score all slightly higher than the other pipelines. Although it performs very close to the pipeline using basic cleaning alone, vocabulary normalization performed by stemming has improved model generalization.

On the other hand, performance degradation was observed in pipelines that added stopword removal. This suggests that removing stopwords in short and informal texts may result in the loss of important information, as they contribute to context. Pipelines using lemmatization also provide moderate performance, but they are slightly inferior to stemming. This suggests that stemming is sufficient to capture the lexical patterns in this dataset, even though lemmatization is computationally advanced.

These results are further analyzed in Section IV-B and confirm that the choice of preprocessing has a direct impact on model performance.

Accordingly, a combination of basic cleaning, auxiliary verb removal, and stemming was chosen as the final preprocessing pipeline. This pipeline helps to reduce noise and preserve meaningful textual patterns, improving classification performance.

B. Target Label Transformation

The original dataset was a multi-label emotion classification problem, where a single text could belong to multiple emotions at the same time. Since the GoEmotions dataset has 27 emotion categories and a neutral class, direct multi-label classification is complex and computationally heavy. Therefore, the main goal of this study is to create a simple and interpretable sentiment classification framework.

To achieve this, the original emotion labels were converted into three main sentiment categories: positive, negative, and ambiguous. A predefined sentiment mapping dictionary was utilized to generate this grouping, with each emotion label assigned to a corresponding sentiment category.

- **Positive:** amusement, excitement, joy, love, desire, optimism, caring, pride, admiration, gratitude, relief, approval
- **Negative:** fear, nervousness, remorse, embarrassment, disappointment, sadness, grief, disgust, anger, annoyance, disapproval
- **Ambiguous:** realization, surprise, curiosity, confusion

Since a single text sample can have multiple emotions, a majority-based decision strategy was used in the target label generation process. The sentiment distribution of emotions in a single text was calculated, and the sentiment category with the highest count was chosen as the final label. In some cases, when a tie occurred between sentiment categories, those samples were considered as mixed classes. To improve labelling consistency in the dataset and lessen model complexity, neutral and mixed samples were removed. The neutral class does not show typical emotional content and therefore does not contribute to the classification boundary. The mixed class was also removed because it increases model noise due to ambiguous labelling. Finally, sentiment classes were transformed into a form suitable for machine learning models using numerical encoding. The label encoding was done as ambiguous = 0, positive = 1, and negative = 2.

Additionally, experiments were conducted by retaining the neutral and mixed sentiment classes in the dataset. However, this resulted in significantly lower performance across all evaluation metrics due to increased ambiguity and class overlap. Therefore, removing neutral and mixed samples was confirmed as a more effective strategy for improving model performance and class separability.

Algorithm 1 illustrates the procedure used for transforming multi-label emotion annotations into a simplified sentiment classification framework.

Algorithm 1 Pseudocode for Target Preprocessing

```
for each sample row IN Dataset D do
    active_emotions ← all emotions where row[emotion] == 1
    if active_emotions is empty then
        sentiment ← "neutral"
        continue
    end if
    sentiment_counts ← {positive: 0, negative: 0, ambiguous: 0}
    for each emotion e IN active_emotions do
        if e IN positive then
            sentiment_counts["positive"] += 1
        else if e IN negative then
            sentiment_counts["negative"] += 1
        else if e IN ambiguous then
            sentiment_counts["ambiguous"] += 1
        end if
    end for
    sentiment ← class with maximum value in sentiment_counts
    if tie exists in sentiment_counts then
        sentiment ← "mixed"
    end if
    ASSIGN row.sentiment = sentiment
end for
REMOVE all rows where sentiment IN {neutral, mixed}
ENCODE: ambiguous → 0, positive → 1, negative → 2
return processed dataset
```

This target transformation process transformed the multi-label emotion detection problem into a simple multi-class sentiment classification problem, which can be efficiently solved using classical machine learning models.

C. Feature Extraction

Text-based data must be converted into a numerical form for use in machine learning models. Two main feature extraction methods suitable for classical machine learning were used: Bag-of-Words (BoW) and Term Frequency–Inverse Document

Frequency (TF-IDF). These methods convert preprocessed text into a vector representation.

1) Bag-of-Words (BoW): In the Bag-of-Words (BoW) model, each piece of text is represented as a vector based on the frequency of words. The order or grammatical structure of the words is not taken into account. Each unique word in the corpus is considered a feature, and a value is assigned based on the number of times that word appears in the text. While this method is straightforward and efficient, the quality of representation may sometimes suffer, as it does not consider the importance or context of the words. The model complexity was reduced by limiting the vocabulary size to 5000 features and removing low-frequency words.

2) Term Frequency-Inverse Document Frequency (TF-IDF): TF-IDF is a feature representation method that improves BoW. It calculates the importance of words based on the number of times that word appears in a document (Term Frequency) and the inverse probability of that word appearing in the entire corpus (Inverse Document Frequency). TF-IDF reduces the influence of common words and increases the weight of words that are important for emotion classification. Therefore, TF-IDF representation is often more discriminative compared to BoW.

3) Feature Representation Setup: Both Bag-of-Words (BoW) and TF-IDF vectorization methods were extended by incorporating n-gram features with a range of $(1, 3)$, allowing the models to capture unigram, bigram, and trigram patterns. This helps to better represent short and informal text by including local word context. Maximum feature size restriction and minimum document frequency filtering were used to reduce the dimensionality of the feature space. Additionally, low-variance features were removed using Variance Thresholding after vectorization. This helped improve model generalization and reduce noise.

D. Model Training

Several classical machine learning models were used for text-based emotion classification. Logistic Regression, Random Forest, XGBoost, SGD, and K-Nearest Neighbours (KNN) algorithms were used as models.

Preprocessed text data were first transformed into numerical feature vectors using the Bag-of-Words (BoW) and TF-IDF methods. Then, these features were separated into training and testing sets using a stratified split method to reduce the effect of class imbalance.

All models were trained on the same feature representations, and their performance was compared using the metrics of accuracy, precision, recall, and F1-score. Here, the macro F1-score was used as the main evaluation metric.

Finally, the performance differences of the models with the feature extraction methods of BoW and TF-IDF are analysed in the Results and Discussion section.

In addition, class imbalance in the dataset was addressed using the Synthetic Minority Over-sampling Technique (SMOTE), which generates synthetic samples for minority classes to improve class distribution, despite its known limitations in sparse text domains. SMOTE was applied only to the training set after data splitting to prevent data leakage. Experiments were also conducted using K-Fold cross-validation to improve model robustness. However, K-Fold cross-validation produced slightly lower performance compared to a stratified train-test split, likely because it provides a more robust and less optimistic estimate of model generalization. Therefore, final results are reported based on the stratified train-test split.

E. Evaluation Metrics

Several commonly used classification metrics were used to evaluate the performance of emotion classification models. Specifically, the models were compared using the metrics of Accuracy, Precision, Recall, and F1-score. When dealing with a dataset that has class imbalance, it is more suitable to use macro-averaged metrics.

Accuracy measures the number of correct predictions relative to the total number of samples:

Precision measures the proportion of cases where the model predicted positive results that were actually positive:

Recall measures the proportion of samples that the model correctly identifies out of the truly positive ones:

F1-Score is the harmonic mean between Precision and Recall, and is used to measure the balanced performance of the model:

The macro-averaged F1-score was specifically used as the main evaluation metric. It is considered a more reasonable and reliable measure for multi-class classification problems with class imbalance. Together with the values of Accuracy, Precision, and Recall, these metrics were used to comprehensively analyze the model performance.

IV. Results

This section presents the results of exploratory data analysis (EDA) and machine learning model experiments conducted on the GoEmotions dataset in detail. This analysis examines the nature of the dataset, emotion distribution, text characteristics, and the relationship between preprocessing and model performance.

A. Exploratory Data Analysis (EDA)

As a first step, exploratory data analysis (EDA) was performed to understand the structure and emotion distribution pattern of the dataset. The GoEmotions dataset is a multi-label classification dataset, and a single text sample can belong to multiple emotions at the same time.

1) Emotion Label Distribution: According to the observations we got from analyzing Fig. 1 of label distribution, we can see the neutral class dominates while making up a large portion of the dataset. Here, many emotions like grief, relief and pride have very low counts, which might require the class balancing techniques or careful metric selection during modelling. In this bar plot, we were able to visualize a long-tail distribution, showcasing that a few emotions are very frequent while many others are rare. While multi-label relationships exist, some likely co-occur with neutral or high-frequency

labels. Also, Fig. 1 highlights the class imbalance and can guide the preprocessing and evaluation strategy.

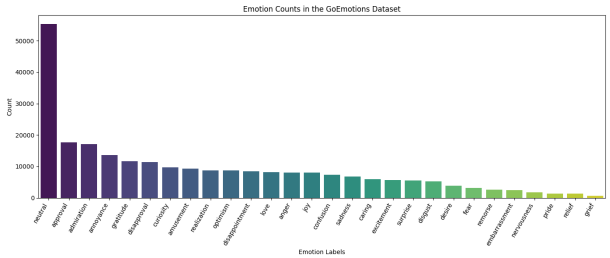

Fig. 1. Emotion Label Distribution.

According to Fig. 2, we can observe that most texts have only one emotion. Fig. 2 also shows that multi-label samples are relatively rare, but there are some cases with up to 12 emotions, indicating some texts are emotionally very rich and ambiguous. In Fig. 2, we can also observe a long-tail distribution, and this distribution will impact model design as models need to handle both single and multi-label cases effectively.

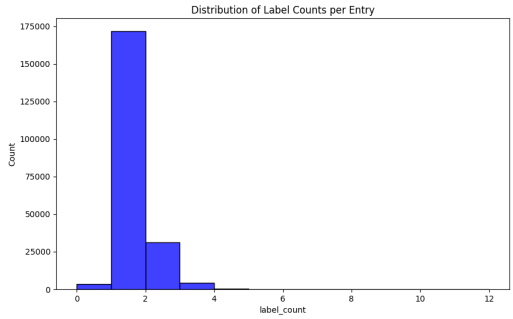

Fig. 2. Distribution of Label Counts per Entry.

2) Sentiment Class Distribution: After simplifying the 27 fine-grained emotions, the dataset was transformed into sentiment categories of positive, negative, ambiguous, neutral, and mixed. According to Fig. 3 shown below, we can observe that the dataset remains moderately imbalanced, even after reducing the original label space. Here, positive sentiment is the most frequent class, followed by neutral and negative sentiments. In Fig. 3, we can see that ambiguous and mixed sentiments are minority classes with significantly fewer samples compared to the main categories. The presence of a large neutral class shows a substantial portion of non-specific or weak emotional content in the dataset. Even though we have simplified the label, the distribution still shows a long-tail effect, where a few classes dominate while others are underrepresented. Even though there is a large portion of records with neutral and mixed sentiment, to improve the model focus and reduce ambiguity, we planned to remove neutral and mixed. After applying this filtering step, the dataset size remains at approximately 140,000 samples, which is still sufficiently large for a robust model.

3) Text Length Distribution: According to the analysis done on the distribution of lengths, we were able to get these

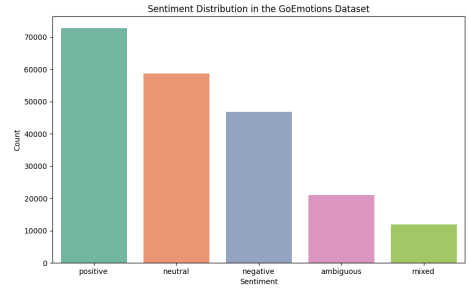

Fig. 3. Sentiment Distribution of the Dataset

observations, and Fig. 4 showcases these. Fig. 4 shows that most texts are short with a length below 100 characters. There are some longer texts that stand out as outliers with lengths reaching 400, 500, or even 700 text lengths. We can visualize these outliers using Fig. 5. Also, Fig. 5 confirms that most texts are condensed below 100 characters, while a few long samples create a long-tail distribution according to Fig. 4. Using the observations we got from this, we can infer that most models must effectively handle short texts efficiently. Also, extremely long texts may require truncation or special preprocessing to avoid skewing the learning process. The long-tail nature of text may also affect padding in vectorization or sequence-based models.

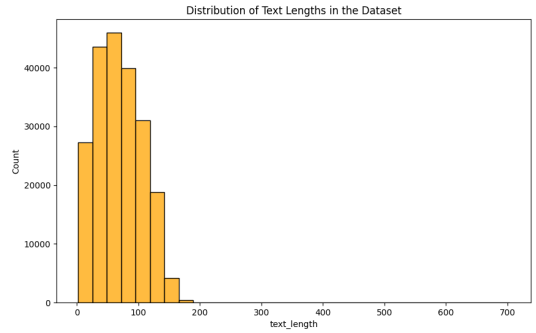

Fig. 4. Distribution of Text Lengths

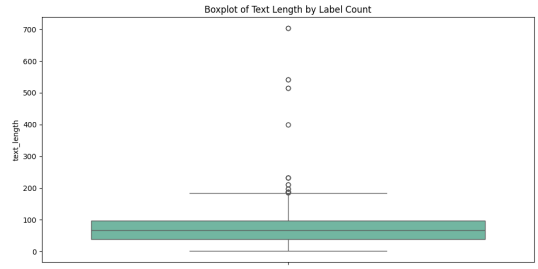

Fig. 5. Boxplot of Text Length by Label Count

4) Word Count and Vocabulary Analysis: Here, we have conducted an analysis of word count and vocabulary usage. In here, Fig. 6 shows that most texts have 10–12 unique words, which is the peak of the distribution. Also, according to Fig. 7, most texts are condensed between 7 and 18 words, confirming that most samples are relatively short and use a

limited vocabulary. A few texts have very high numbers of unique words (up to 32–33), representing long or complex texts, forming a long-tail distribution in Fig. 6. According to the implications we got from these observations, most texts are concise, so models must efficiently handle short inputs. Outlier texts with many unique words may require truncation or special preprocessing to avoid skewing feature vectors. The limited number of unique words in texts impacts feature sparsity in vectorization methods like Bag-of-Words or TF-IDF.

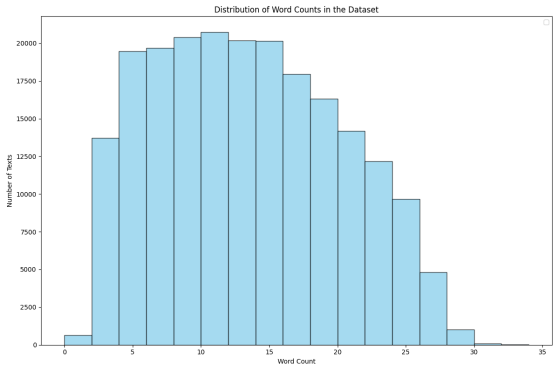

Fig. 6. Distribution of Word Counts

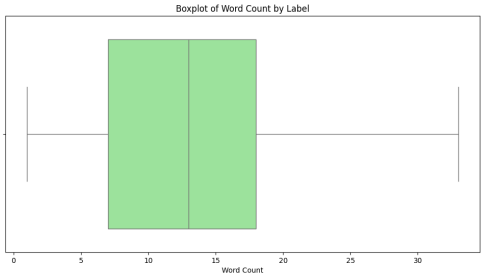

Fig. 7. Boxplot of Word Count by Label

Fig. 8 showcases that many common words in texts are general conversational words such as ‘like’, ‘im’, ‘dont’, ‘thats’, ‘one’, and ‘would’. Some words may indicate emotions or interactions, such as ‘love’, ‘good’, ‘lol’, and ‘thanks’. This highlights that the dataset contains informal, conversational text with many short, casual phrases. These frequent words may dominate feature vectors, so careful processing or weighting may help improve modelling.

5) Emotion Correlation Analysis: According to Fig. 9, we can see a positive correlation between similar emotions. For examples:

- Joy and Excitement: 0.10
- Nervousness and Fear: 0.20
- Annoyance and Anger: 0.09
- Sadness and Disappointment: 0.09

This indicates that similar or related emotions tend to appear together, which is expected in multi-label texts.

Also, from Fig. 9, we can visualize that most emotions have negative correlation with neutral. As examples:

Fig. 8. Word Cloud for Common Words

- Admiration and Neutral: -0.18
- Annoyance and Neutral: -0.16
- Gratitude and Neutral: -0.14

And there are many values around -0.10, -0.11, -0.12, -0.13. This makes sense because texts labeled as neutral generally do not express other emotions, so they rarely co-occur.

According to observations, positive correlations are generally low but meaningful, suggesting some co-occurrence patterns exist. Neutral tends to be mutually exclusive with other emotions. Other emotion pairs mostly have weak correlation expressions.

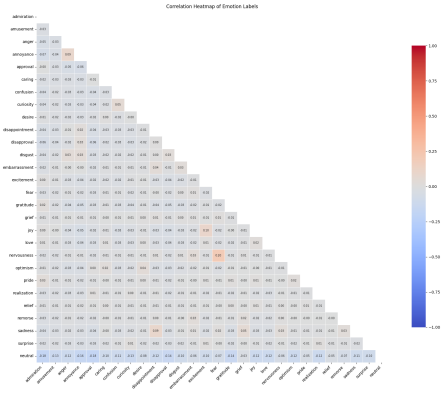

Fig. 9. Correlation Heatmap Between Emotion Labels

B. Preprocessing Results

In this section, the performance of various text preprocessing pipelines was compared. The aim of this is to identify the most suitable preprocessing method for emotion classification.

Here, Basic Cleaning, Stopword Removal, Stemming, Lemmatization, and several combinations of them were evaluated using TF-IDF vectorization and a Random Forest model. Table I shows the performance results (Accuracy, Precision, Recall, F1-score) of these pipelines.

Key Observations:

- The stemming pipeline (Basic Cleaning + Stemming) achieved the highest performance with an F1-score of 0.6943
- Using Basic Cleaning alone also showed very similar performance
- Lemmatization showed slightly lower performance than Stemming

- Pipelines using stopword removal showed a consistent drop in performance
- This further confirms that stopwords contribute to contextual meaning in short texts

TABLE I
PREPROCESSING PIPELINE COMPARISON

Pipeline	Acc.	Prec.	Rec.	F1
Stemmed	0.7439	0.6930	0.6958	0.6943
Basic	0.7431	0.6919	0.6945	0.6931
Lemmatized	0.7427	0.6915	0.6943	0.6928
No Stopwords + Stemmed	0.7410	0.6869	0.6952	0.6907
No Stopwords + Lemmatized	0.7378	0.6837	0.6928	0.6878
No Stopwords	0.7370	0.6823	0.6917	0.6865

Analysis: According to these results, stemming helps normalize vocabulary and improves model generalization even when using richer n-gram features. Although the improvements are marginal, the stemming pipeline consistently outperforms other methods. The use of stopword removal leads to performance degradation, reinforcing that contextual words are important for emotion classification in short and informal text.

Final Selection: Based on the results of this experiment, the "Basic Cleaning + Stemming" preprocessing pipeline was selected for the final model. This pipeline provided a good balance between noise reduction and meaningful information preservation, and achieved the highest F1-score, as shown in Table I.

C. Model Performance

We compared the performance results obtained using different machine learning models and feature extraction methods (BoW and TF-IDF). All models were trained under the same preprocessing pipeline (Basic Cleaning + Stemming) and evaluated using the metrics of accuracy, precision, recall and macro F1-score. These results are shown in detail in Table II.

Key Observations:

- The Random Forest + TF-IDF combination achieved the highest performance (F1 ≈ 0.6943)
- Random Forest also performed similarly well with BoW features
- KNN showed moderate improvement compared to previous experiments but still underperformed compared to ensemble methods
- Logistic Regression and SGD provided stable but lower performance
- XGBoost showed moderate performance but did not outperform Random Forest

Analysis: The use of n-gram features $(1, 3)$ and SMOTE helped improve contextual representation and class balance. However, their impact on overall performance was not consistently positive, suggesting that synthetic sampling may introduce noise affecting generalization. Random Forest continues to perform best due to its robustness with high-dimensional sparse features. Unlike earlier results, TF-IDF slightly outperforms BoW, indicating that term weighting becomes more beneficial when combined with n-gram features.

TABLE II
MODEL PERFORMANCE COMPARISON

Model	Vec.	Acc.	Prec.	Rec.	F1
Random Forest	TF-IDF	0.7439	0.6930	0.6958	0.6943
Random Forest	BoW	0.7429	0.6920	0.6941	0.6928
KNN	TF-IDF	0.7159	0.6674	0.6878	0.6706
KNN	BoW	0.7148	0.6658	0.6889	0.6710
XGBoost	BoW	0.7153	0.6680	0.6504	0.6578
Logistic Reg.	BoW	0.7099	0.6569	0.6744	0.6635
SGD	BoW	0.7019	0.6504	0.6738	0.6576
Logistic Reg.	TF-IDF	0.7009	0.6510	0.6767	0.6584
XGBoost	TF-IDF	0.6853	0.6414	0.6590	0.6438
SGD	TF-IDF	0.6805	0.6349	0.6635	0.6409

Final Outcome: According to the final results of this experiment, the "TF-IDF + Random Forest" combination was identified as the best-performing approach. This was confirmed to be the most suitable approach for emotion classification tasks, providing high accuracy and a balanced F1 score.

D. Confusion Matrix Analysis

The confusion matrix of the best-performing model (Random Forest with TF-IDF) is shown in Fig. 10. The model achieves strong correct classifications for the positive (5877) and negative (3440) classes, indicating good learning of clear sentiment patterns.

However, the ambiguous class shows the highest misclassification rate, with many samples being predicted as positive (446) or negative (505). This suggests that ambiguous texts often share overlapping features with other classes.

Some confusion is also observed between positive and negative classes, reflecting contextual ambiguity in short social media texts.

Overall, the results confirm that while the model performs well on clear sentiments, distinguishing ambiguous expressions remains the main challenge.

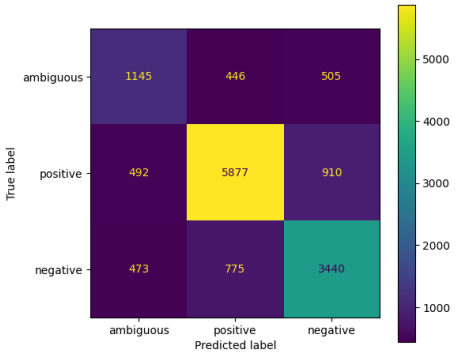

Fig. 10. Confusion Matrix for Random Forest + TF-IDF Combination

V. Discussion & Limitations

The results of this study show that the three components of preprocessing, feature representation, and model selection are crucial for text-based emotion classification. In particular, the highest performance (Accuracy ≈ 0.744, F1 ≈ 0.694) was achieved by using the Random Forest model with the Basic Cleaning + Stemming preprocessing pipeline and the TF-IDF feature representation.

The introduction of n-gram features and SMOTE improved class balance and enhanced contextual representation. However, these techniques did not consistently improve overall performance and may introduce noise that affects generalization. However, despite these improvements, the overall performance slightly decreased compared to earlier experiments. This suggests that while SMOTE improves class distribution, it may introduce synthetic noise that affects generalization.

Additionally, experiments conducted with K-Fold cross-validation resulted in slightly lower performance compared to the stratified train-test split. This is likely because cross-validation provides a more robust and less optimistic estimate of model generalization.

Furthermore, retaining neutral and mixed classes significantly reduced model performance, confirming that these classes introduce ambiguity and reduce class separability.

This shows that classical machine learning approaches can provide good performance even without complex deep learning methods, especially for short and noisy social media text data. The TF-IDF representation gives slightly better results than BoW, indicating that even simple lexical features are sufficient for emotion classification.

Also, according to the preprocessing results, it was observed that using stemming simplifies the feature space and improves model performance, while stopword removal may reduce performance by losing contextual information. EDA analysis also revealed that features such as class imbalance, short-text nature, and weak emotion correlations in the dataset affect model performance.

However, this study has several limitations. First, the original multi-label problem is simplified to sentiment-based classification (positive, negative, and ambiguous). This simplifies modelling but results in the loss of fine-grained emotional information. Second, the class imbalance in the dataset is not fully addressed, and the underrepresentation of rare emotion classes may introduce model bias.

Another limitation is the lack of context-aware models (e.g., deep learning models such as BERT). Classical models are limited in recognizing semantic relationships, making it difficult to recognize complex language patterns such as sarcasm and irony. Also, only BoW and TF-IDF are used as feature extraction methods, and advanced representations such as word embeddings are not considered.

In conclusion, although this study presents a practical and computationally efficient solution, future work could further improve performance by using deep learning approaches, class balancing techniques, and multi-label modeling.

VI. Conclusion

The main objective of this study was to identify the appropriate preprocessing, feature extraction, and machine learning model combination for text-based sentiment classification. Initially considered a multi-label emotion classification problem, it was simplified to positive, negative, and ambiguous sentiment categories to reduce complexity.

Exploratory Data Analysis (EDA) identified features of the dataset such as class imbalance, long-tail distribution, and short-text nature. These observations were used to guide preprocessing and modeling decisions.

Preprocessing experiments confirmed that the Basic Cleaning + Stemming pipeline provided better performance. Among the Bag-of-Words (BoW) and TF-IDF methods used for feature extraction, the TF-IDF method was found to provide slightly better results.

According to the model comparison results, the Random Forest + TF-IDF combination achieved the highest performance (Accuracy ≈ 0.744, F1-score ≈ 0.694). This shows that simple lexical features and classical machine learning methods can provide effective and computationally efficient solutions for short and noisy social media text data.

The use of n-gram features and SMOTE improved class balance and contextual representation, although it introduced a slight decrease in overall performance compared to earlier configurations.

Overall, this study shows that the relationship between preprocessing, feature representation, and model selection is very important in the sentiment classification task. This study confirms that by choosing the right combination, a computationally efficient and practical solution can be obtained.

References

[1] S. Poria, E. Cambria, N. Howard, G.-B. Huang, and A. Hussain, "Text-based emotion recognition using deep learning approach," *Wireless Communications and Mobile Computing*, vol. 2022, pp. 1–13, 2022.

[2] A. Pasad, A. Singh, and P. K. Atrey, "Emotion detection in text: A review," *arXiv preprint arXiv:1806.00674*, Jun. 2018.

[3] H. Nobari et al., "A review on text-based emotion detection—Techniques, applications, datasets, and future directions," *arXiv preprint arXiv:2205.03235*, Apr. 2022.

[4] Y. Wang et al., "Emotion recognition of social media users based on deep learning," *PeerJ Computer Science*, vol. 9, p. e1414, Jun. 2023.

[5] D. García-Pablos, A. Cuadros, M. Aparicio, and A. Agerri, "Emotion classification for short texts: An improved multi-label perspective," *Humanities and Social Sciences Communications*, vol. 10, no. 1, p. 307, Jun. 2023.

[6] J. Li et al., "Improving multilabel text emotion detection with emotion relationship graphs," *SSRN Electronic Journal*, Jan. 2025.

[7] Y. Zhang et al., "Bridging contextual gaps in text-based emotion detection using large language models," in *Proceedings of SemEval*, 2025.

[8] D. Demir et al., "GoEmotions: A Dataset of Fine-Grained Emotions," *arXiv preprint arXiv:2005.00547*, 2020.

[9] S. Ramakrishnan, "Improving Multi-Label Emotion Classification on Imbalanced Datasets using BERT," in *Proc. IEEE Conf. Computational Intelligence*, 2025, pp. 1–6.

[10] J. Song, "Fine-Grained Emotion Detection on GoEmotions," *arXiv preprint arXiv:2601.18162*, Jan. 2026.

[11] A. Kumar and S. Sharma, "Comparative Analysis of Text based Emotion Detection on GoEmotions Dataset," in *Proc. IEEE Int. Conf. Innovative Computing (ICIC)*, 2023, doi: 10.1109/ICIC59316.2023.10541692.